

Laboratorio de Estrés para el Monitoreo de Recursos en Linux utilizando Docker y Jupyter Notebook

Kevin Santiago Palomino Murcia, Juan Manuel Polania Navarro

Curso Sistemas Operativos, Programa de Ingeniería en Sistemas, Facultad de Ingeniería,
Universidad del Valle seccional Tuluá

Abstract

En este trabajo se realizó un laboratorio de estrés con el objetivo de analizar cómo se comporta un sistema operativo Linux cuando varios servicios utilizan al mismo tiempo los recursos del computador. Para las pruebas se utilizó Ubuntu WSL2 junto con Docker, donde se ejecutó una aplicación web desarrollada en Next.js, una base de datos PostgreSQL, PgAdmin y un entorno de Jupyter Notebook para entrenar un modelo de inteligencia artificial. Durante el desarrollo del proyecto se realizaron diferentes pruebas de carga sobre el procesador, la memoria RAM y la base de datos, mientras se monitoreaba el sistema con herramientas como **htop**, **docker stats** y **pg_stat_activity**. Los resultados mostraron que el entrenamiento del modelo de inteligencia artificial fue el proceso que más recursos consumió, especialmente memoria RAM y procesador. Además, al ejecutar todas las pruebas al mismo tiempo se observó una disminución en el rendimiento del sistema, permitiendo identificar los procesos que generaban mayor carga y los principales cuellos de botella. Con este proyecto fue posible comprender la importancia del monitoreo de recursos y del uso de herramientas que ayudan a analizar el comportamiento del sistema operativo bajo condiciones de alta demanda.

Keywords: Linux, Docker, PostgreSQL, monitoreo de recursos, estrés del sistema, Jupyter Notebook.

I. Introducción

Actualmente muchas aplicaciones funcionan utilizando varios servicios al mismo tiempo, como servidores web, bases de datos y herramientas de análisis de datos. Todos estos servicios comparten los recursos del sistema, principalmente el procesador y

la memoria, por lo que es importante conocer cómo se comportan cuando existe una alta carga de trabajo.

En este proyecto se construyó un entorno de pruebas utilizando Docker sobre Ubuntu WSL2. Dentro de este entorno se ejecutaron varios servicios de manera simultánea: una aplicación web desarrollada con Next.js, una base de datos PostgreSQL, una herramienta de administración llamada PgAdmin y un entorno de trabajo con Jupyter Notebook.

El objetivo principal fue observar cómo responde el sistema cuando diferentes procesos compiten por los mismos recursos. Para ello se generaron cargas de trabajo sobre la aplicación web, la base de datos y un modelo de inteligencia artificial, mientras se monitoreaba el consumo de recursos utilizando herramientas del sistema operativo.

Este artículo presenta la metodología utilizada para realizar las pruebas de estrés, los resultados obtenidos durante el monitoreo del sistema y el análisis de los cambios observados en el consumo de recursos cuando varios procesos se ejecutan al mismo tiempo.

II. Marco Teórico

2.1 Gestión de procesos en Linux

Linux es un sistema operativo que permite ejecutar varios procesos al mismo tiempo. Cada proceso utiliza recursos como el procesador, la memoria RAM y el almacenamiento para poder funcionar correctamente. Cuando varios procesos se ejecutan de forma simultánea, el sistema operativo se encarga de administrar estos recursos para que todos puedan trabajar de la mejor manera posible. Si la cantidad de procesos aumenta demasiado, el rendimiento del sistema puede disminuir debido a la alta demanda de recursos.

2.2 Docker y la virtualización ligera

Docker es una herramienta que permite ejecutar aplicaciones dentro de contenedores. Cada contenedor tiene todo lo necesario para que una aplicación funcione sin afectar las demás. A diferencia de una máquina virtual, los contenedores comparten el mismo sistema operativo, por lo que consumen menos recursos y son más rápidos de iniciar. En este proyecto se utilizó Docker para ejecutar la aplicación web, PostgreSQL, PgAdmin y Jupyter Notebook de forma independiente.

2.3 Monitoreo de recursos

Para conocer el estado del sistema durante las pruebas se utilizaron diferentes herramientas de

monitoreo. htop permitió observar el uso del procesador, la memoria RAM y los procesos activos en tiempo real. docker stats mostró el consumo de recursos de cada contenedor, mientras que las herramientas de monitoreo de PostgreSQL permitieron revisar la actividad de la base de datos y el número de conexiones activas durante las pruebas.

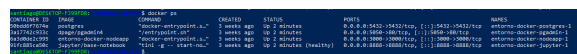
2.4 Pruebas de estrés

Las pruebas de estrés consisten en aumentar la carga de trabajo de un sistema para observar cómo responde cuando utiliza una gran cantidad de recursos. En este proyecto se generó carga sobre la aplicación web, la base de datos y el entrenamiento de un modelo de inteligencia artificial. Esto permitió analizar el comportamiento del sistema cuando varios procesos compiten al mismo tiempo por el procesador y la memoria RAM.

III. Metodología

3.1 Entorno de pruebas

Las pruebas se realizaron sobre Ubuntu WSL2 utilizando contenedores Docker. El entorno estuvo compuesto por una aplicación web desarrollada con Next.js, una base de datos PostgreSQL, PgAdmin para la administración de la base de datos y Jupyter Notebook para la ejecución de código de inteligencia artificial.



CONTAINER ID	IMAGE	COMMAND	STATUS	PORTS	IP	NAME
dockerps-01	postgres	postgres -c 'max_wal_size=1GB'	Up 2 minutes	5432/tcp	172.17.0.2	postgres
dockerps-02	pgadmin/pgadmin4	pgadmin4	Up 2 minutes	5050/tcp	172.17.0.3	pgadmin4
dockerps-03	nextjs	next start	Up 2 minutes	3000/tcp	172.17.0.4	nextjs
dockerps-04	jupyterlab	jupyter lab	Up 2 minutes	8888/tcp	172.17.0.5	jupyterlab

Figura 1. Contenedores Docker activos.

Cada servicio se ejecutó dentro de su propio contenedor, permitiendo simular un entorno similar al que se utiliza en aplicaciones reales. Todos los contenedores compartieron los recursos disponibles del sistema anfitrión.

3.2 Procedimiento experimental

El experimento se desarrolló en tres etapas. En la primera etapa se verificó que todos los servicios estuvieran funcionando correctamente antes de iniciar las pruebas. Para ello se comprobó el estado del sistema, el funcionamiento de Jupyter Notebook, el

acceso a PgAdmin y la conexión con la base de datos PostgreSQL.

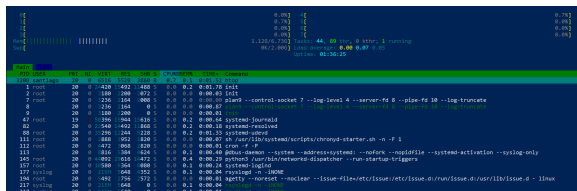


Figura 2. CPU en reposo antes de la carga.

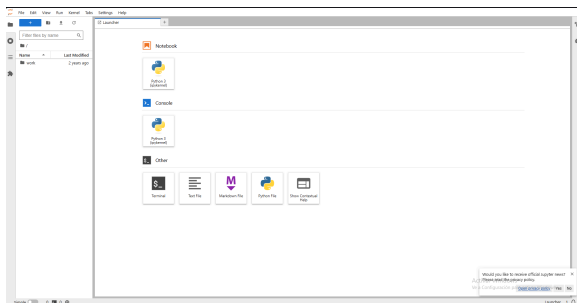


Figura 3. JupyterLab operativo.

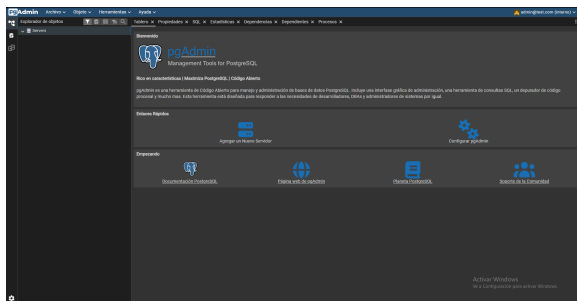


Figura 4. PgAdmin operativo.

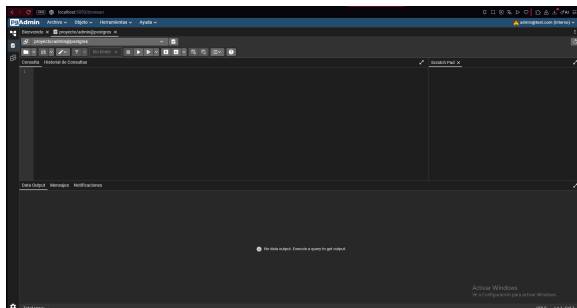


Figura 5. Conexión exitosa a PostgreSQL.

En la segunda etapa se aplicaron pruebas de carga sobre la base de datos mediante inserciones y

consultas repetitivas para observar el comportamiento de PostgreSQL.

Finalmente, en la tercera etapa se ejecutó el entrenamiento de un modelo de inteligencia artificial en Jupyter Notebook mientras la aplicación web y la base de datos continuaban en funcionamiento. De esta forma fue posible analizar el comportamiento del sistema cuando varios servicios utilizaron los recursos del computador al mismo tiempo.

3.3 Herramientas de monitoreo

Para monitorear el comportamiento del sistema se utilizaron diferentes herramientas. **htop** permitió observar el uso del procesador, la memoria RAM y los procesos activos en tiempo real. **docker stats** se utilizó para revisar el consumo de recursos de cada contenedor, mientras que las herramientas de monitoreo de PostgreSQL permitieron verificar la actividad de la base de datos y el número de conexiones durante las pruebas.

IV. Resultados

4.1 Carga sobre PostgreSQL

Para aumentar la actividad de PostgreSQL se realizaron inserciones masivas de registros con el fin de generar una carga significativa sobre la base de datos.

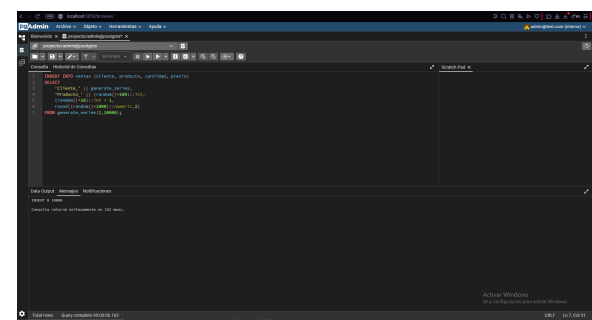


Figura 6. Inserción masiva de registros.

Durante la ejecución de consultas repetitivas se observó un incremento en la utilización del procesador.

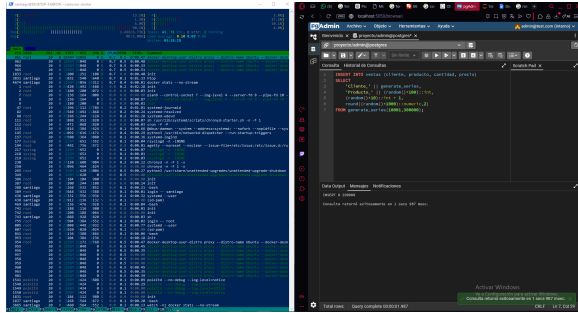


Figura 7. Uso de CPU durante consultas PostgreSQL.

El monitoreo mediante docker stats permitió visualizar el comportamiento de los contenedores durante las pruebas.

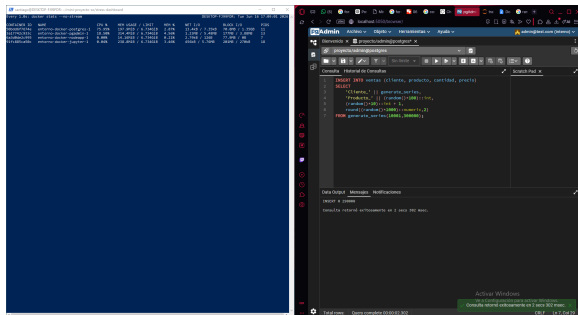


Figura 8. Docker Stats durante consultas PostgreSQL.

Además, PostgreSQL registró un aumento en el número de conexiones activas mientras se ejecutaban múltiples consultas de manera simultánea.

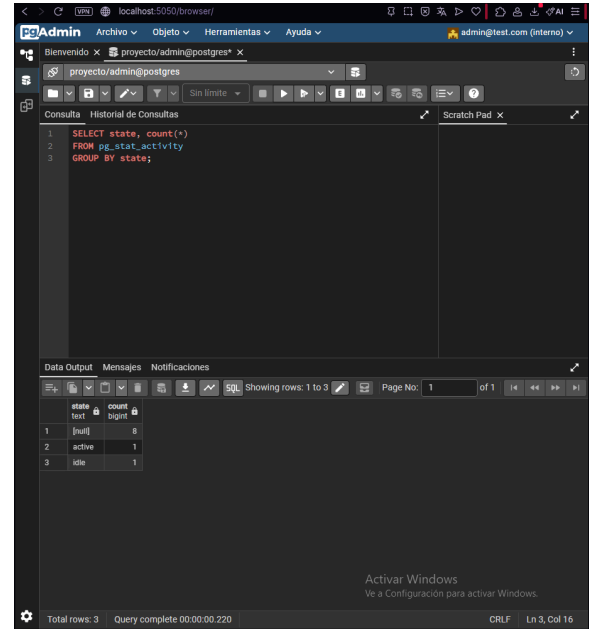


Figura 9. Conexiones activas PostgreSQL.

4.2 Entrenamiento del modelo de inteligencia artificial

Antes de iniciar el entrenamiento del modelo se configuró el entorno de trabajo dentro de Jupyter Notebook y se prepararon las herramientas necesarias para el monitoreo.

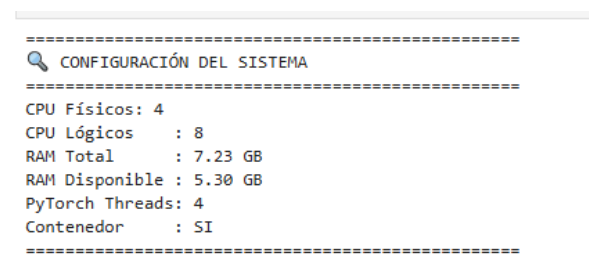


Figura 10. Configuración del sistema en Jupyter.

Posteriormente se generó el conjunto de datos utilizado para producir carga sobre el procesador y la memoria.

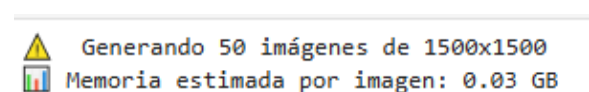


Figura 11. Generación del dataset de estrés.

Una vez preparado el entorno, se cargó el modelo neuronal utilizado para las pruebas.

 Parámetros del modelo: 1,090,939,018
 Memoria estimada del modelo: 4.36 GB

Figura 12. Modelo neuronal cargado.

Durante la ejecución del entrenamiento se produjo la terminación del kernel por agotamiento de memoria (OOM), lo que evidenció la alta demanda de recursos del sistema.

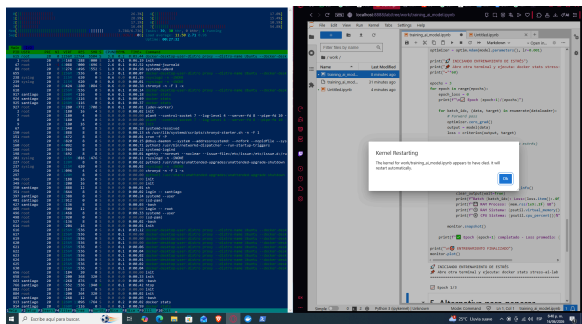
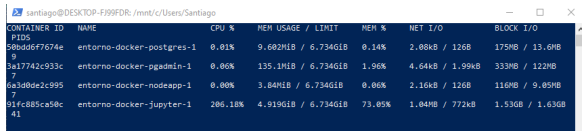


Figura 13. Terminación del kernel por agotamiento de memoria (OOM).

Después se observó el consumo de recursos de los contenedores durante el entrenamiento del modelo.

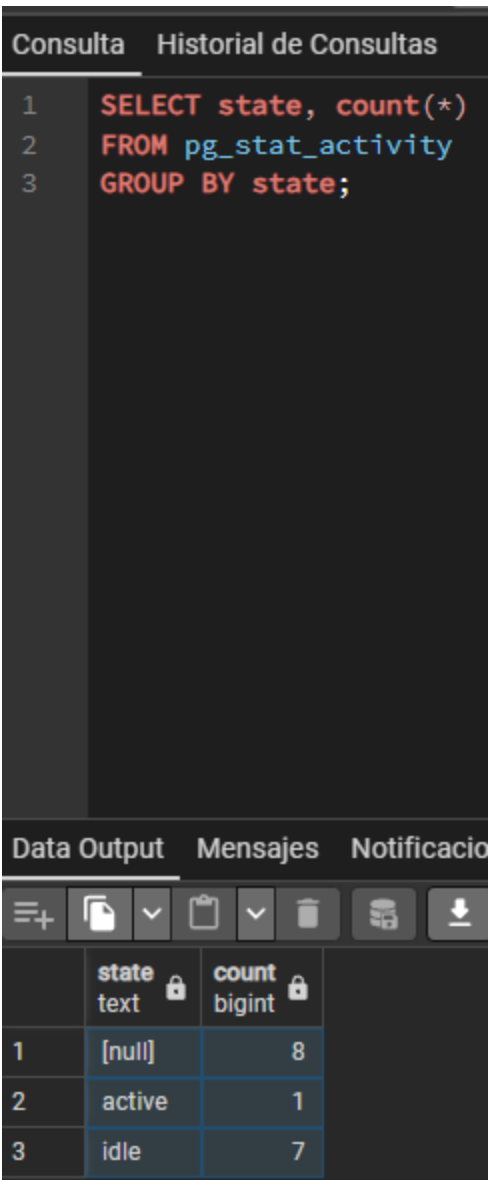


CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O
9b0d0e2c995	entorno-docker-postgres-1	0.01%	9.602MiB / 6.734GiB	0.14%	2.08kB / 126B	175MB / 13.6MB
9b0d0e2c995	entorno-docker-pgadmin-1	0.06%	135.1MiB / 6.734GiB	1.96%	4.64kB / 1.99kB	333MB / 122MB
9b0d0e2c995	entorno-docker-nodeapp-1	0.00%	3.64MiB / 6.734GiB	0.06%	2.16kB / 126B	116MB / 9.05MB
9b0d0e2c995	entorno-docker-jupyter-1	206.10%	4.919GiB / 6.734GiB	73.05%	1.04MB / 772kB	1.53GB / 1.63GB

Figura 14. Consumo de recursos de contenedores durante el entrenamiento.

4.3 Carga concurrente

Finalmente, se ejecutaron de manera simultánea la aplicación web, las operaciones sobre PostgreSQL y el entrenamiento del modelo de inteligencia artificial. En esta etapa se registró el mayor nivel de utilización de recursos y una evidente disminución del rendimiento general del sistema.



Consulta **Historial de Consultas**

```
1 SELECT state, count(*)
2 FROM pg_stat_activity
3 GROUP BY state;
```

Data Output **Mensajes** **Notificación**

	state 	count 
1	[null]	8
2	active	1
3	idle	7

Figura 15. Estado inestable bajo carga concurrente.

Durante las pruebas se observó un aumento progresivo en el uso de los recursos del sistema. Al generar múltiples solicitudes sobre la aplicación web se evidenció un incremento en el uso del procesador, especialmente por parte de los procesos asociados a Node.js.

Las consultas e inserciones realizadas sobre PostgreSQL aumentaron la actividad de la base de datos y el número de conexiones activas. Esto pudo observarse tanto en las herramientas de monitoreo

como en las estadísticas proporcionadas por PostgreSQL.

Por otra parte, la ejecución del modelo de inteligencia artificial produjo un consumo considerable de memoria RAM y procesador. Este comportamiento fue más evidente durante las etapas de entrenamiento, donde el sistema tuvo que procesar grandes cantidades de datos.

Cuando todas las cargas se ejecutaron simultáneamente se observó una disminución general del rendimiento. El sistema continuó funcionando, pero presentó una mayor utilización de recursos y una respuesta más lenta en algunos servicios.

V. Análisis

Los resultados obtenidos muestran que los diferentes servicios compitieron por los recursos del sistema cuando se ejecutaron al mismo tiempo. La aplicación web desarrollada en Next.js aumentó principalmente el uso del procesador debido a la cantidad de solicitudes realizadas, mientras que PostgreSQL presentó un incremento en el número de consultas y conexiones activas.

El proceso que generó la mayor carga fue el entrenamiento del modelo de inteligencia artificial ejecutado en Jupyter Notebook. Durante esta prueba se observó un alto consumo de CPU y memoria RAM, llegando incluso a producir la terminación del kernel por agotamiento de memoria (OOM). Esto permitió identificar que la memoria RAM fue el principal cuello de botella durante las pruebas más exigentes.

El comportamiento del sistema se monitoreó utilizando herramientas como **htop**, **docker stats** y las herramientas de monitoreo de PostgreSQL. Gracias a estas herramientas fue posible observar el consumo de CPU, memoria RAM y la actividad de los diferentes servicios, identificando los momentos en los que el sistema presentó la mayor carga.

Para recuperar el funcionamiento normal del sistema se detuvieron las pruebas de carga sobre la aplicación web, PostgreSQL y el entrenamiento del modelo de

inteligencia artificial. Después de esto, el consumo de CPU y memoria volvió a niveles normales, permitiendo que los servicios continuaran funcionando correctamente y confirmando que el alto consumo de recursos estaba asociado a las pruebas de estrés realizadas.

VI. Conclusiones

A partir de las pruebas realizadas fue posible analizar el comportamiento de un sistema Linux ejecutando varios servicios de manera simultánea dentro de contenedores Docker.

Se comprobó que el uso intensivo de la aplicación web incrementa el consumo del procesador y que PostgreSQL aumenta considerablemente su actividad cuando recibe múltiples consultas e inserciones.

También se observó que el entrenamiento de modelos de inteligencia artificial requiere una gran cantidad de memoria RAM y capacidad de procesamiento, convirtiéndose en uno de los procesos que más recursos consume dentro del sistema.

Las herramientas de monitoreo utilizadas permitieron identificar los momentos de mayor carga y los principales cuellos de botella, facilitando el análisis del comportamiento de los diferentes servicios durante las pruebas.

Como limitación del proyecto, las pruebas se realizaron utilizando los recursos disponibles del computador donde se desarrolló el laboratorio, por lo que los resultados pueden variar al ejecutarse en equipos con diferentes características de hardware.

Como trabajo futuro, sería interesante realizar pruebas con equipos de mayor capacidad, incluir más servicios ejecutándose al mismo tiempo y comparar los resultados utilizando otros entornos de virtualización o diferentes configuraciones de Docker.

En general, este proyecto permitió comprender mejor cómo un sistema operativo administra sus recursos cuando varios procesos se ejecutan al mismo tiempo

y la importancia de utilizar herramientas de monitoreo para detectar problemas de rendimiento.

Referencias

[1] Docker Inc., Docker Documentation, 2025.

[2] PostgreSQL Global Development Group, PostgreSQL Documentation, 2025.

[3] Node.js Foundation, Node.js Documentation, 2025.

[4] Linux Foundation, Linux System Monitoring Tools, 2024.

[5] PyTorch Foundation, PyTorch Documentation, 2025.

[6] Project Jupyter, Jupyter Notebook Documentation, 2025.